Kodewala Academy(1st June, 2026) - Java Backend Test # 7

Daksh Arya
Aug. 24, 2026, 12:01 p.m.

You've survived the test. The results will arrive soon. 😎

Your score: 92% 27.5 / 30
Duration: 0:15:04

1. Which statement is correct? — 1 / 1 point

Your Answer: ✔ Correct
○ throw is used to declare an exception.
○ throws actually throws the exception.
✔ ● throw is used to explicitly throw an exception object.
○ Both are identical.

2. — 1 / 1 point

```
try {
    int x = 10 / 0;
}
catch (RuntimeException e) {
    System.out.print("A");
}
catch (ArithmeticException e) {
    System.out.print("B");
}
```

What will happen?

Your Answer: ✔ Correct
○ A
○ B
○ AB
✔ ● Compilation Error

3. — 1 / 1 point

```
try {
    System.out.println("try");
    return;
} finally {
    System.out.println("finally");
}
```

What will happen?

Your Answer: ✔ Correct
○ Only try
○ Only finally
✔ ● try followed by finally
○ Compilation error

4. — 1 / 1 point

```
static int test() {
    try {
        return 10;
    } finally {
        return 20;
    }
}
System.out.println(test());
```

What is the output?

Your Answer: ✔ Correct
○ 10
✔ ● 20
○ Runtime exception
○ Compilation error

5. Which statement is correct regarding checked exceptions? — 1 / 1 point

Your Answer: ✔ Correct
○ They must always be caught
○ They must always be declared with throws
✔ ● They must be handled or propagated
○ They cannot be created manually

6. What happens if a finally block throws an exception while the try block also throws one? — 1 / 1 point

Your Answer: ✔ Correct
○ Both exceptions are automatically propagated
○ Exception from try always wins
○ Compilation error
✔ ● Exception from finally can replace/suppress the original exception

7. Which exception is unchecked? — 1 / 1 point

Your Answer: ✔ Correct
○ IOException
○ SQLException
○ ClassNotFoundException
✔ ● NullPointerException

8. — 1 / 1 point

```
try {
    throw new Exception("A");
} catch (Exception e) {
    System.out.println(e.getMessage());
}
```

What is the output?

Your Answer: ✔ Correct
○ null
✔ ● A
○ Exception
○ Compilation error

9. — 1 / 1 point

```
String a = "Ja";
String b = "va";
String c = "Java";
String d = a + b;
System.out.println(c == d);
```

What is the output?

Your Answer: ✔ Correct
○ true
✔ ● false
○ Compilation error
○ Runtime Exception

10. — 1 / 1 point

```
String a = "Ja" + "va";
String b = "Java";
System.out.println(a == b);
```

What is the output?

Your Answer: ✔ Correct
✔ ● true
○ false
○ Compilation error
○ Runtime Exception

11. — 1 / 1 point

```
String s = "Java";
s.replace("a", "o");
System.out.println(s);
```

What is the output?

Your Answer: ✔ Correct
○ Jovo
✔ ● Java
○ Javo
○ Runtime Exception

12. — 1 / 1 point

```
class Parent {
    void show() {
        System.out.println("Parent");
    }
}
class Child extends Parent {
    void show() {
        System.out.println("Child");
    }
}
Parent p = new Child();
p.show();
```

What is the output?

Your Answer: ✔ Correct
○ Parent
✔ ● Child
○ Compilation error
○ Runtime Exception

13. Can constructors be overridden? — -0.25 / 1 point

Your Answer: ✘ Incorrect
✘ ● YES
✔ ○ NO
○ Only protected constructors
○ Only default constructors

14. Which method cannot be overridden? — 1 / 1 point

Your Answer: ✔ Correct
○ public method
○ protected method
✔ ● final method
○ abstract method

15. Which statement about method overriding is correct? — 1 / 1 point

Your Answer: ✔ Correct
○ Static methods are dynamically dispatched
○ Private methods can be overridden
○ Final methods can be overridden
✔ ● Return type can be covariant

16. What happens here? — -0.25 / 1 point

```
class A {
    private void show() {
        System.out.println("A");
    }
}
class B extends A {
    void show() {
        System.out.println("B");
    }
}
```

Your Answer: ✘ Incorrect
○ B overrides A's method
○ Compilation error
✘ ● B defines a separate method
✔ ○ Runtime exception

17. Which concept hides implementation details while exposing necessary functionality? — 1 / 1 point

Your Answer: ✔ Correct
○ Inheritance
✔ ● Abstraction
○ Polymorphism
○ Overloading

18. Can an abstract class have a constructor? — 1 / 1 point

Your Answer: ✔ Correct
✔ ● YES
○ NO
○ Only private constructor
○ Only public constructor

19. What can cause a memory leak in Java? — 1 / 1 point

Your Answer: ✔ Correct
○ Creating objects inside a method
○ Using int variables
○ Calling System.out.println()
✔ ● Keeping unnecessary objects referenced in a collection

20. Which collection allows duplicate elements and maintains insertion order? — 1 / 1 point

Your Answer: ✔ Correct
○ HashSet
✔ ● ArrayList
○ Set
○ TreeSet

21. — 1 / 1 point

```
Set<Integer> set = new HashSet<>();
set.add(10);
set.add(20);
set.add(10);
System.out.println(set.size());
```

Your Answer: ✔ Correct
○ 1
✔ ● 2
○ 3
○ Compile time error

22. Which statement about HashSet is correct? — 1 / 1 point

Your Answer: ✔ Correct
○ It guarantees insertion order
○ It guarantees sorted order
✔ ● It does not guarantee iteration order
○ It allows duplicate elements

23. What happens when add() is called on a Set with an existing element? — 1 / 1 point

Your Answer: ✔ Correct
○ Duplicate is added
○ Existing element is replaced
✔ ● add() returns false
○ Exception is always thrown

24. Which operation is generally expensive in the middle of an ArrayList? — 1 / 1 point

Your Answer: ✔ Correct
○ get(index)
✔ ● Inserting/removing elements in the middle
○ Reading the first element
○ set(index, value)

25. What happens if you insert objects into a TreeSet without natural ordering and without providing a Comparator? — 1 / 1 point

Your Answer: ✔ Correct
○ Objects are always accepted
○ Objects are stored randomly
✔ ● A ClassCastException can occur
○ Objects are automatically sorted by hashCode

26. — 1 / 1 point

```
List<String> list = new ArrayList<>();
list.add("Java");
list.add("Spring");
list.add("Java");
Set<String> set = new HashSet<>(list);
System.out.println(set.size());
```

What is the output?

Your Answer: ✔ Correct
○ 1
✔ ● 2
○ 3
○ Compilation error

27. — 1 / 1 point

```
class Employee {
    int id;
    Employee(int id) {
        this.id = id;
    }
    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Employee)) return false;
        return id == ((Employee) o).id;
    }
    @Override
    public int hashCode() {
        return id;
    }
}
```

What happens if you execute the below code?

```
Employee e = new Employee(10);
Set<Employee> set = new HashSet<>();
set.add(e);
e.id = 20;
System.out.println(set.contains(e));
```

Your Answer: ✔ Correct
○ true
✔ ● false
○ NullPointerException
○ Compilation error

28. — 1 / 1 point

```
import java.util.HashSet;
import java.util.Set;
class Student {
    int id;
    Student(int id) {
        this.id = id;
    }
    @Override
    public boolean equals(Object obj) {
        return obj instanceof Student &&
            this.id == ((Student) obj).id;
    }
    @Override
    public int hashCode() {
        return super.hashCode();
    }
}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(10);
        Student s2 = new Student(10);
        Set<Student> set = new HashSet<>();
        set.add(s1);
        set.add(s2);
        System.out.println(set.size());
    }
}
```

Your Answer: ✔ Correct
○ Runtime Exception
○ 1
✔ ● 2
○ Compilation error

29. — 1 / 1 point

```
class Product {
    int id;
    Product(int id) {
        this.id = id;
    }
    @Override
    public boolean equals(Object obj) {
        return obj instanceof Product &&
            id == ((Product)obj).id;
    }
    @Override
    public int hashCode() {
        return 100;
    }
}
Set<Product> set = new HashSet<>();
set.add(new Product(1));
set.add(new Product(2));
System.out.println(set.size());
```

Your Answer: ✔ Correct
○ Runtime Exception
○ 1
✔ ● 2
○ 0
○ Compile error

30. — 1 / 1 point

```
class Person {
    int id;
    Person(int id) {
        this.id = id;
    }
    @Override
    public boolean equals(Object obj) {
        return true;
    }
    @Override
    public int hashCode() {
        return id % 2;
    }
}
```

What is the output if you run below code?

```
Set<Person> set = new HashSet<>();
Person p1 = new Person(10);
Person p2 = new Person(12);
Person p3 = new Person(14);
set.add(p1);
set.add(p2);
set.add(p3);
System.out.println(set.size());
```

Your Answer: ✔ Correct
○ Runtime Exception
✔ ● 1
○ 2
○ 3
○ Compile error

Logout